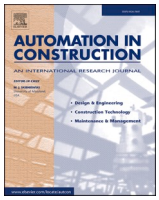
Automation in Construction 166 (2024) 105622

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon



Automated construction scheduling using deep reinforcement learning with valid action sampling

Yuan Yao, Vivian W.Y. Tam *, Jun Wang, Khoa N. Le, Anthony Butera

Western Sydney University, School of Engineering, Design and Built Environment, Locked Bag 1797, Penrith, NSW 2751, Australia

ARTICLE INFO

Keywords:
Construction scheduling
Reinforcement learning
Action masking
Reward shaping
Graph neural network
Off-site construction

ABSTRACT

The growing demand for buildings and infrastructures requires optimal construction schedules under real-world complexities. This paper presents an automated scheduling optimization method for large construction projects, considering real-world constraints and the need for rescheduling. A Deep Reinforcement Learning (DRL) model with a Valid Action Sampling (VAS) mechanism is proposed to optimize schedules. The method integrates a Graph Convolutional Network (GCN) for feature extraction and includes a reward shaping mechanism to expedite convergence. The proposed method outperforms traditional methods with reduced project duration and runtime in both scheduling and rescheduling cases. This advancement benefits construction managers seeking efficient and flexible project management. The findings inspire future research into broader and more practical construction scheduling solutions utilizing DRL.

1. Introduction

Construction scheduling is a method that helps project managers make better decisions regarding labour, equipment, time, cost, etc. As a decision-making process, construction scheduling determines which tasks should be completed and defines how and when these tasks should be carried out. The scheduling problem can be regarded as Resource Constrained Project Scheduling Problem (RCPSP) [13]. In a typical constructing schedule, the overall tasks are divided into different Work Breakdown Structures (WBS) and activities which will be assigned to corresponding sub-contractors [8]. In real-world construction projects, the scheduling of tasks is complicated as the sequence of activities is constrained by various factors such as labour, resources, and construction processes [21,28]. The construction constraints can be roughly divided into two categories: precedence constraints and resource constraints [10,42]. Critical and proper scheduling plans will provide a clear and constraint-free workflow for all activities and contractors on their construction site, greatly enhances efficiency and reduce the total cost. To complete a construction schedule, an experienced scheduler with in-depth prior knowledge should handle all information from the construction site carefully and such a process is time-consuming [5]. Another practical need for project managers is to typically adjust schedules based on real-world conditions, such as evaluating the benefits of adjusting the quantities of labour or equipment resources [35]. These tasks require an automated scheduling method with a certain level of rescheduling ability, meaning the ability to reapply existing strategies to produce schedules under different conditions.

Many methods and theories have been introduced into construction scheduling. One of the most widely accepted methods is the Critical Path Method (CPM) which breaks projects into different WBS and then reorganizes them manually [1]. However, CPM method assumes that resources are unlimited, which in most cases does not align with reality [39]. Soman and Molina-Solana [40] presented a simple case study involving a 12-activity project subject to resource constraints. In this scenario, CPM with manual correction required 15 min. To avoid constraint conflicts and make a better schedule, researchers have been searching for many ways to help automatically generate well-organized schedules. Kolodner [26] introduced the case-based reasoning (CBR) methods, which can adapt and use former cases to analyse and solve new cases. Such methods rely on historical experience and are not flexible to cope with different types of construction projects. Other methods including heuristic approaches and Genetic Algorithm (GA) define the scheduling problem as an optimization problem with single or multiple objectives subjected to constraints [17]. Such methods are less able to generalise to different types of construction projects and are not able to capture the uncertainties during the construction process. To mitigate the complexity of scheduling, many studies have leveraged deep learning (DL) for automated scheduling. A notable study by Hong et al.

* Corresponding author.
E-mail address: vivianwytam@gmail.com (V.W.Y. Tam).

https://doi.org/10.1016/j.autcon.2024.105622
Received 13 December 2023; Received in revised form 24 June 2024; Accepted 9 July 2024
Available online 30 July 2024

[19] introduced a Graph-Based Automated Scheduling (GAS) method, which utilized a Graph Convolutional Network (GCN) to extract sequence information from historical schedules. This information was then used to generate an optimal schedule through GA. GCN enabled their work can be used in different cases to obtain precedence information. However, deep learning methods depend on a robust historical dataset, which is challenging to procure. The data they collected was not necessarily optimal, requiring further optimization of the generated schedules by GA. It should be noted that the optimized schedule generated by the GA is only applicable to the initial conditions at the start of optimization. When the initial resource configuration changes, the GA needs to be recalculated, which means the GA lacks the rescheduling ability.

As a popular decision-making problem solution, Deep Reinforcement Learning (DRL) has been applied to solve scheduling optimization problems with complex constraints [36]. In the DRL paradigm, the agent learns to obtain the optimum action policy, such as the optimum activity sequence, by exploiting rewards in the environment through a delayed reward mechanism [41]. In the field of construction scheduling, such rewards can be an objective that relates to the construction duration or total cost. To use traditional algorithms such as GA, it is necessary to set different heuristic rules for different tasks; and iteration is needed to optimize the schedule. However, DRL doesn't need such customized settings and iteration, it automatically learns different optimization strategies during the training and simulation process. Compared to other artificial intelligence (AI) methods, DRL does not require large datasets to provide historical data for training, which also reduces the algorithm development and implementation difficulty. Since DRL and other AI application in construction scheduling is an emerging area, there is limited research in this field. Recent work by Soman and Molina-Solana [40] introduced DRL methods into the domain of construction look-ahead scheduling (LAS). But their method failed to make the use of precedence of construction activities and presented a complex action space. Kadir et al. [25] developed a reinforcement learning-graph embedding network and adopted Graph Convolution Network (GCN) to extract the topological precedence information of the project. However, they did not simplify the construction topology, which makes their method complex with more construction activities. However, their study provided a high-dimension action space, which make the training of DRL difficult. Furthermore, their works also ignored the problems of regenerating the schedule for schedule evaluation. Their studies assumed the construction with a determined resource configuration. Utilizing pre-trained DQN as a rescheduling strategy was not discussed in current studies.

Motivated by previously studies, this paper investigates the implementation of DRL to solve the resources constrained construction scheduling problem and validate the proposed model with major real-world bridge construction projects. This study proposes a Deep Q-Network (DQN) based constrained scheduling method that can be used as a decision-making tool in construction scheduling by generating constraints-free and optimal scheduled workflows.

The contributions of this study are as follows:

1. This study proposes a DQN-based method that can automatically generate a constraint-free and optimized schedule considering resource utilization and manpower allocation. The generated detailed schedule can guide a construction manager to organize the various resources and subcontractors on site.
2. This study innovatively proposes a modified DQN architecture with a Valid Action Sampling (VAS) mechanism, which enhances the model's capacity for processing complex and large construction projects. The VAS can narrow the range of action combinations and provides the DQN network with a small and fixed output dimension, which reduces the DQN complexity. This mechanism provides a concise and feasible algorithmic structure for construction projects that involve hundreds of activities.
3. Reward shaping is adopted and discussed in the DQN design. Well-designed reward representation will help the agent distinguish the potential actions' quality. With reward shaping, the agent can find the optimal action quickly without falling into local optimum. This enables a faster convergence DQN, which could help the scheduler decide the optimal schedules in a short time.
4. The proposed DQN method demonstrates rescheduling ability under different resource settings. A pre-trained DQN can be directly utilized to generate schedules. Faster rescheduling can be used to help schedulers quickly evaluate schedules under varying initial resource conditions.

2. Related works

2.1. Scheduling in construction

The concept of construction project scheduling caught researchers' attention in the 1990s [16]. As a decision-making process, construction project scheduling aims to make optimal activity sequences and assign various resources to a set of tasks under heterogeneous constraints [50]. Using mathematical algorithms, researchers introduced resource optimisation systems for the allocation and levelling of limited construction resources. Christodoulou [12] proposed an agent-based ant colony optimization (ACO) method to solve resource-constrained construction projects. Cheng et al. [11] introduced a symbiotic organisms search (SOS) method in solving multiple project scheduling problems. Verbeeck et al. [43] used Artificial Immune method with local search procedure to solve a time-constrained project scheduling problem. El-Rayes and Jun [14] eliminate undesirable resource fluctuations and resource idle time via a GA method. Bettemir et al. [9] used a hybrid GA method to make an optimal construction schedule in a large case study with 120 activities. Ahmed et al. [2] introduced a computer-aided simulation modelling-based approach to reduce the total completion time and optimize resource utilization. Xie et al. [45] developed a GA method considering prefabricated component supply constraints. The results showed that such a method can effectively reduce completion time and improve project performance. Yuan et al. [48] designed a Hybrid Cooperative Co-evolution Algorithm to generate a schedule for prefabricated building (PB) construction projects. Erdal and Kanit [15] developed software based on the GA approach which made the construction project more sustainable by minimizing project duration and optimizing resource allocation. Ma et al. [32] considered the construction sequence of different PC and CiS components; they proposed multi-objective discrete symbiotic organisms search to shorten the makespan. Lin et al. [28] proposed an input-adaptive particle swarm optimization model to solve the dynamic situations of resource-constraint problems, introducing a novel approach to RCPSP. Considering the financial limits, Liu et al. [30] developed a heuristic-based GA to help construction managers achieve expected profits and implement project control.

With the above thorough research on heuristic and meta-heuristic method applications in construction scheduling, limited studies have been made on the other field of methods such as AI methods [42]. Amer et al. [4] used Transformer Model to predict the long-term and short-term schedule plans. Hong et al. [19] discussed and proposed GCN methods which extract the sequence information from labelled data; hence they used GA to generate optimal schedules. Amer [3] implement Implicit Logic Checker method with Transformer Decoder for implicit constraints prediction. They tested their method in a case with 7 activities and obtained an optimal schedule with shorter duration.

2.2. Rationale

DRL is an area of AI that is initially inspired by the learning ability of animals in nature. It simulates the response of the animal, namely the agent, to the received environmental feedback reward after taking an

action. A typical DRL consists of three elements: 1) agent(s) with a set of actions $\widehat{A}$; 2) a set of states S; 3) a set of rewards r (Fig. 1). In DRL methods, the agent aims to maximize the total reward by interacting with the dynamic environment through making optimal decisions [41].

In DRL-based scheduling, the agent represents the project managers who are responsible for determining the activity sequences with no resource constraint violation. The design form of action can be directly related to construction activities, or it can be designed to indirectly choose activities by selecting certain rules. For instance, actions could be defined as executing a single construction activity [25], carrying out multiple activities simultaneously [40], or choosing different heuristic rules or constituent algorithms [38,49]. For objectives focused on controlling capital flow, actions might involve determining the quantity of workers and the number of resources to purchase [23]. The environment needs to be developed that can reflect the construction schedule or schedule-related information. Such an environment could be realized as a construction progress simulator, where all construction activities and resources are built-in and can interact with each other. The states represent the on-site project condition including project progress and labour allocation. Similar to action, rewards should align with these objectives. Applying negative rewards to the duration of the schedule and positive rewards for higher resource utilization rates are examples of potential reward designs.

The construction scheduling process can be described as a sequential decision-making process due to its complex, dynamic, and stochastic nature. In the real-world construction process, there is typically a specific sequence to follow, where a clear precedence among construction activities exists. Additionally, as the construction progresses, the feasible activities and external constraints also change accordingly. Therefore, the process of creating a schedule for a construction project is essentially about sequencing a multitude of construction activities. This process aligns with the core mechanism of DRL, which aims to learn a policy from a series of states and actions [34]. Consequently, DRL could be effectively utilized to optimize construction schedules.

Another advantage of the DRL method is that it does not require the creation of a comprehensive dataset [24,27]. Unlike other deep learning methods that require large amounts of data for supervised learning, DRL learns through trial-and-error interactions within a digital simulation environment. Various constraints are incorporated as built-in coefficients in the simulation environment to ensure that the environment can accurately replicate the real construction process. DRL makes decisions on actions, specifically determining which activities should be executed, and receives feedback from the environment based on these actions. The construction simulator executes the activities and provide new construction states back to the DRL. Then DRL decides which construction activities to undertake next based on the feedback received. By continuously simulating the entire construction process, DRL learns to maximize positive feedback and thus derives an optimal construction schedule. For DRL application in construction schedule optimization, the only requirement for the model training is a customized environment with detailed project information, which is easier to achieve than collecting dataset.

The integration of deep neural networks with DRL has empowered it to process and learn from high-dimensional inputs, thereby equipping it to navigate the intricacies inherent in real-world scenarios [33]. These neural networks function as sophisticated approximators, adept at managing the large-scale or high-dimensional state spaces often encountered in practical problems. Consequently, DRL can abstract knowledge from unprocessed sensory data and formulate optimal strategies, even in contexts where conventional methodologies may be impeded by the curse of dimensionality. Fundamentally, the role of neural networks in DRL is pivotal for distilling valuable features from complex inputs, thus bolstering the learning process by facilitating pattern recognition and the implementation of informed decisions based on the policy derived through learning. The dynamic nature of construction processes, which evolve at distinct junctures in response to current activities and their progression, results in a variety of state features. These features encompass a spectrum of details that are inherently complex and multidimensional, including the overall progress of the project, the specific activities being executed, and the deployment of labour and machinery. Deep neural networks possess the ability to discern and extract subtle features from these variable states, thereby enabling DRL to judiciously select the most fitting construction activities, optimizing the efficacy and progression of the project.

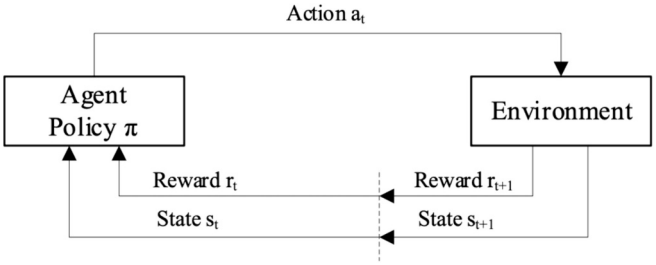


Fig. 1. DRL framework.

2.3. DRL approaches in scheduling

With the ability to solve complex decision-making problems, DRL methods have been introduced to the field of Construction Engineering and Management (CEM). One of the popular application domains of DRL methods is Building Energy Management and Control. Ye et al. [47] developed a DQN-based method for real-time energy management under uncertain household demands. Gupta et al. [18] introduced DRL methods into heating control regarding the dynamic indoor-outdoor temperature. In the building maintenance domain, studies were also carried out. Some work was applied to bridge maintenance planning [6,44]. Some studies had focused on the long-term maintenance planning of pavements [37,46]. In the building design area, Liu et al. [31] proposed a multi-agent model-free DRL method to generate reinforcement bars design. These studies show that DRL methods can be applied to solve CEM problems. However, reinforcement learning has not yet been applied in-depth on a large scale in construction scheduling [7].

Soman and Molina-Solana [40] considered combining multiple activities, which simplified the impact of complex projects on DRL training. However, their approach required a full permutation of all possible combinations, resulting in an exponential growth in the DRL agent's action space with project complexity, which in turn increased the neural network's parameter count and complexity. For instance, if the quantity of construction process is 10, the size of action space would grow to 2^{10}, which will cause difficulty for neural network training. Their work failed to introduce mechanism that can reduce such complexity. Additionally, their work did not directly eliminate the impact of constraint violations during the training process, which also reduces the training efficiency of DQN. The reward design of their work considered constraint violation, activity completion and recourses cost. However, the constraint violation is unnecessary and can be avoided by proper action space design. Kedir et al. [25] proposed a GCN-DRL and defined each construction activity as an individual node in the construction graph, leading to a significant increase in the number of nodes for complex projects, which directly affects the performance of the GCN. Their GCN method can be reapplied to different cases. However, their case studies are relatively simple with 25 activities. Recent work by Jiang et al. [23] provided a novel flow-view scheduling model. Their model optimized the labour and cash flow situation during the whole construction process. However, they assumed only three types of onsite activities and corresponding resources, which oversimplified the real-world construction. With the advances in the three mostly related DRL research in construction scheduling, the rescheduling ability of DRL is not discussed in their research. Rescheduling ability refers to the trained

DRL agent can be directly applied in different case settings, such as different initial resources and even different projects. Rescheduling ability makes DRL superior to most traditional meta-heuristic algorithms that need change initial settings and calculate from the beginning.

2.4. Summary and research gap

In summary, based on the literature review, Table 1 presents the most relevant studies along with details of their methodologies. Compared to DRL, GA lacks in terms of interactivity as it does not incorporate real-world construction simulation. Additionally, GA's maximum activity quantities are also lower than those of DRL. And GA algorithm still lacks the rescheduling ability. Theoretically, DRL has the ability to reschedule with different case settings, but current studies do not discuss and validate it. To bridge the gaps in current, this study aims to develop a feasible DQN architecture that can be applied on scheduling real-world construction project by modifying the structure of the neural network. The rescheduling ability is validated by rescheduling experiments with different resource quantities using the trained DRL agent.

3. Research methods

3.1. Problem definition

This study will define a pre-fabric bridge construction scheduling problem as an optimization problem that aims to minimize the total duration subjected to heterogeneous constraints. The construction process consists of a set of various structure groups $SG_i = \{1, 2 \ldots i\}$. These structure groups are constrained in different ways like precedence constraints which allow superstructures to begin only after the substructures are complete. Each structure group consists of various sub structures $r_{i,j} \in SG_i$, $j = \{1, 2 \ldots j\}$. In this research, sub structures are used to define the physical structure which belongs to a structure group. They are different components which are different in construction methods. Sub structures are unique in the way of required subcontractors, equipment, and resources. Activities $a_{i,j,k} \in r_{i,j}$, $k = \{1, 2 \ldots k\}$ are the smallest elements which give direct orders to the subcontractors about the information of equipment and resources required. They divide sub structures into discrete parts constrained by recourses and precedence. Take bridge as example, structure groups in bridge construction can be categorised into 1) superstructure, including the girder and joints; and 2) substructure, including pile, pile cap, pier, and pier cap. The hierarchical relationship primarily considers the craftsmanship and physical constraints. Only after the adjacent two substructures are completed can the corresponding superstructure between them begin construction. This hierarchical relationship is also reflected in the activities required for different segments. The activity-segment-structure group relationship is illustrated in Fig. 2. As the schedulers' interests are to complete the project earlier, the objective of the proposed optimization is to minimize the total duration of a construction project.

3.2. Objective

$$minimize\ D_{project} \tag{1}$$

Subject to:

$$\sum R_t\left(a_{i,j,k}\right) \le R \tag{2}$$

$$s_{k+1} - s_k \ge d_k \tag{3}$$

$$N_c = 0 \tag{4}$$

Where $D_{project}$ denotes the total duration of the project; $R_t\left(a_{i,j,k}\right)$ denotes the resources required for activity $a_{i,j,k}$; $R = [R_1, R_2 \ldots R_l]$ denotes the renewable resources set, R_l represents the quantity of the l^{th} resource; s_k denotes the starting time of the k^{th} activity; d_k represents the duration of k^{th} activity; N_c denotes the number of constraint violations.

Eq.1 indicates the optimization objective in minimizing the total project duration. Eq.2 indicates that at time t, all the assigned resources must not exceed the available resources. Eq.3 means the starting time of the $k+1^{th}$ activity must wait until the k^{th} activity is complete if they are precedence constrained. Eq.4 indicates that there should not be any constraint in the final schedule plan. The goal of this study is to propose an automatic method to generate an optimized schedule in time and cost under heterogeneous constraints.

3.3. DRL model design

3.3.1. DQN architecture

The architecture of the proposed DQN framework is illustrated in Fig. 3. The initial construction information is used to: 1) develop a simulation environment for model training and 2) transfer to a graph where the construction activity sequences are embedded. Standard DQN is required to output a vector representing the Q-values for the entire action space. As the dimensionality of the action space increases, the vector's length increases accordingly, leading to an exponential growth

Table 1
Summary of the most related studies.

References		Method	Objectives	Constraints	Real-world Construction Simulation	Maximum Activity quantity	Rescheduling ability
Meta-heuristic Algorithm	Christodoulou [12]	ACO	F_m	R, P	No	17	No
	Lin et al. [28]	PSO	F_m	R, P	No	61	No
	Ma et al. [32]	DSOS	F_m, C_m	P	No	106	No
	Bettemir et al. [9]	Hybrid GA	F_m	R, P	No	120	No
	Xie et al. [45]	GA	F_m	R, P	No	120	No
	Erdal and Kanit [15]	GA	F_m	R, P	No	14	No
	Hong et al. [19]	GCN + GA	F_m,C_m	R, P	No	21	No
Artificial Intelligence	Soman and Molina-Solana [40]	DRL	F_m, C_m	R, P	Yes	500	Not validated
	Kedir et al. [25]	DRL	F_m	R, P	Yes	25	Not validated
	Proposed study	DRL	$\boldsymbol{F_m}$	R, P	Yes	497	Yes

ACO: Ant Colony Optimization; PSO: Practical Swarm Optimization; DSOS: Discrete Symbiotic Organisms Search; GA: Genetic Algorithm; GCN: Graph Convolution Network; DRL: Deep Reinforcement Learning.
F_m: Minimize duration; C_m: minimize cost.
R: resource constraint; P: precedence constraint.

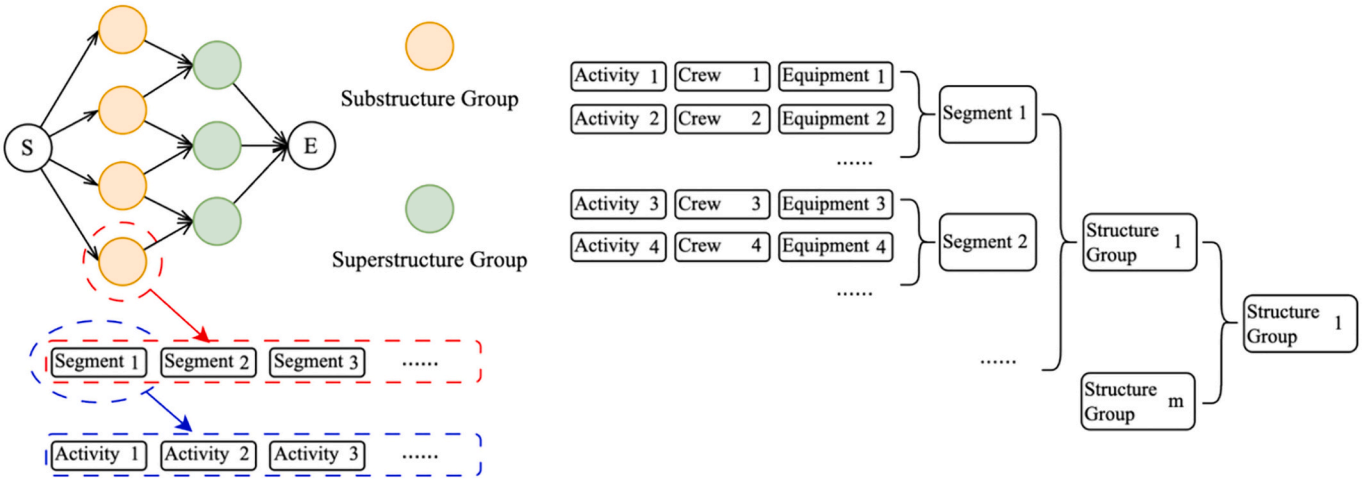


Fig. 2. Activity, segment, and structure group

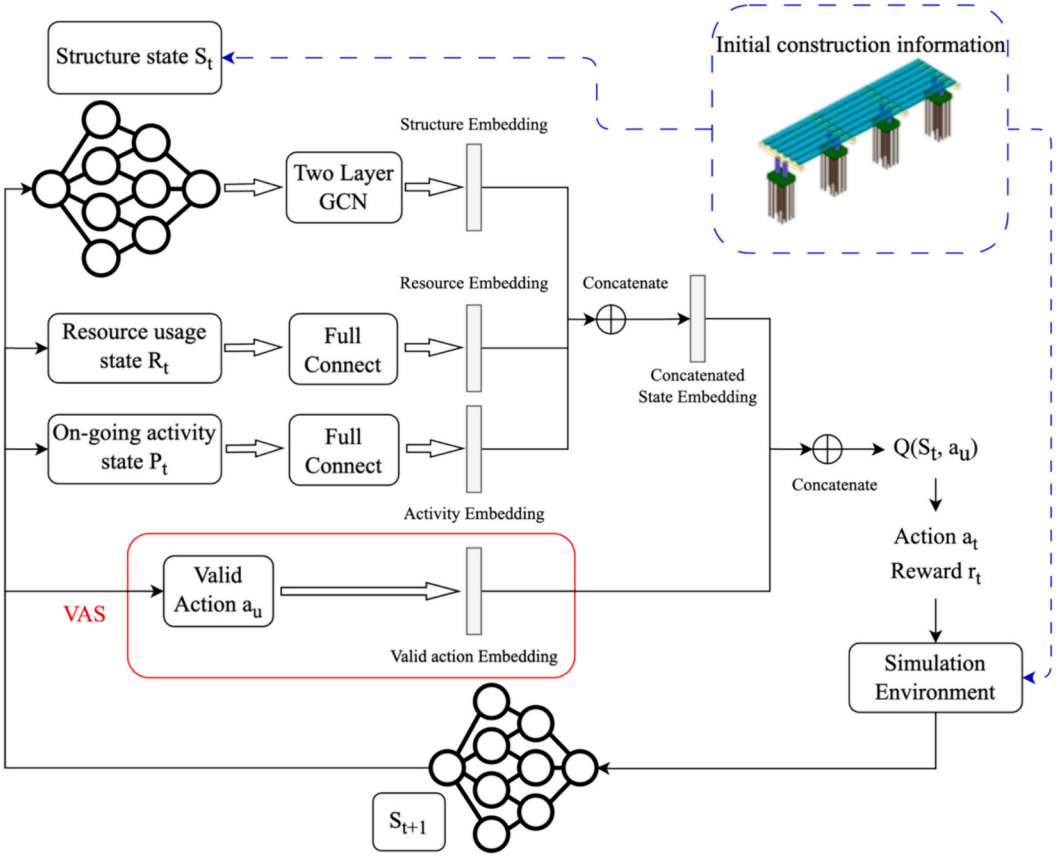


Fig. 3. VAS DRL Architecture.

in the number of parameters within the neural network, which makes the model difficult to train. To address this issue, this study introduces a valid action sampling method (VAS). At each timestep, VAS provides all valid actions for the current state that do not violate any constraints. These valid actions are combined with a concatenated state embedding through an embedding layer, eventually outputting the Q-values for the selected valid actions. Considering the number of valid actions may vary at each timestep, the sampled valid actions are padded to a fixed length with special tokens. As the size of the valid action set is smaller than the high-dimensional action space, the neural network's output layer for computing Q-values can be set to a relatively smaller length, thus reducing complexity, and improving training efficiency. The VAS method is initially motivated by the similar action masking mechanism proposed by [22]. Action masking enables the agent to ignore all other invalid tasks. At each timestep, the mask will provide a series of valid safe action combinations for the DQN agent according to the environment state.

For the DRL agent, choosing an invalid action will return a negative reward and terminate the training episode. Such early termination will stop the agent exploring the possible action, resulting in the low training efficiency and slow reward convergence. In the illustrative example (Fig. 4), there are only 4 valid tasks while the total quantity of all the

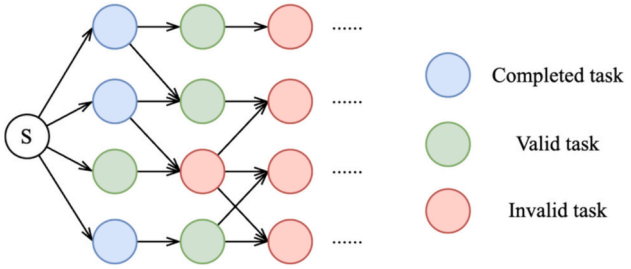


Fig. 4. Valid action after masking.

tasks might be significantly larger. With action masking, the agent shall only focus on the combinations of valid tasks.

3.3.2. States, action, and reward design

The agent action is decided by the deep neural network which represents the highest reward value in every environment state. To let the DQN agent "learn" better, a clear state and action representation is needed.

3.3.2.1. State. On construction sites, the state is the combination of the project information. The state S_t in the DQN method can be represented as follows in Eq.5:

$$S_t = [G, R, P] \tag{5}$$

1) Structure completion state G: The completion state indicates the overall progress of the project. Meanwhile, it determines the precedence constraints as it shows the completeness of the structure groups. This state will guide the DQN agent to choose constraints-free actions.
2) Resource usage state R: Resources serve as the limitation for deciding which action should be executed in the next time step. It contains the quantity of all the resources on site.
3) On-going activities state P: This state gives information on activities in progress. It will assess which activity should be priority selected.

$G = (V, E, H)$ denotes the topological representation of the structure completion state, where V represents the node set, E represents the edge set and H represents the node feature. H is a matrix with m structure groups and n corresponding sub structures (Eq.6).

$$H = \begin{bmatrix} \widehat{H_1} \\ \vdots \\ \widehat{H_m} \end{bmatrix} \tag{6}$$

Where the element $\widehat{H_m} = [s_0, s_1, H_{m,1}, H_{m,2}, \ldots H_{m,n}]$ represents the node feature of m^{th} structure group. The first two elements $[s_0, s_1]$ represent the status of the m^{th} structure group using one hot code method. $[s_0, s_1] = [0, 0]$ means the structure group has been not started. $[0, 1]$ means the structure group has been started. $[1, 1]$ means the structure group has been completed. $H_{m,n}$ denotes the number of completed activities of n^{th} sub structures under m^{th} structure group. The value of $H_{m,n}$ will change as the project is conducted. $H_{m,n} = N_t$ means the corresponding sub structures has finished N_t activities. Initially, when the construction is not started, the value of $H_{m,n} = -1$.

It should be noted that in real-world construction projects, the number of sub structures under each structure group can be different. In such cases, the shape of H will be irregular. Irregular node features will cause dimension errors in proceeding GCN. Thus, the irregular feature matrix should be modified by padding the shorter node feature with $H_{m,n} = -2$. After the padding, all the node features will be the same dimension as the longest feature and the GCN can be processed.

The resource matrix R is a vector with elements indicating the number of k kinds of resources, representing the resource availability at each timestep (Eq.7). The elements indicate the number of corresponding resources such as workers, materials, and equipment.

$$R = [n_{r1}, n_{r2}, \ldots n_{rl}] \tag{7}$$

$R_l = 0$: the l^{th} resource has been used up.
$R_l = n_{rl}$: amount n of the l^{th} resource available.

The activity state matrix P is a vector with k elements in which the elements contain the information of ongoing activities, consisting of the current activities $p_{i,j,k}$, corresponding resources required r_k, and activity progress $d_{i,j,k}$ (Eq.8, Eq.9). When a new activity is chosen, the transition p will be added to activity matrix P, and the progress $d_{i,j,k} = 0$. When the activity is finished, the corresponding a will be deleted from the matrix. It should be noted that as the quantities of on-going activities are changing frequently. To avoid such change, the length of P is determined by a test run without joining P. Then in further training, if the actual quantity of activities is less than the length of P, the missing elements will be padded with -1. Here presents an illustrative example of how the matrix P is determined in real simulation. Let's assume the length of P is set to 4, and at a given moment, there are already 2 ongoing activities. The first number in each row represents the activity ID, the second and third number represent the labour and equipment resource, and the last number represents the duration. At this time, a new activity is added to the execution list. An illustrative example explains that when the number of ongoing activities is less than the length of P, the empty activity positions will be replaced by dummy activities Fig. 5. When a new activity is introduced, it will be sequentially positioned after the existing activities. Similarly, when an activity is completed, it will be removed. For different real-world cases, the length of P could be set according to the actual situation.

$$P = [p_1, p_2, \ldots p_k] \tag{8}$$

$$p_k = \left[p_{i,j,k}, r_k, d_{i,j,k}\right] \tag{9}$$

$p_{i,j,k}$: the k^{th} activity of j^{th} sub structures under i^{th} structure group.
r_k: the resources required of the k^{th} activity.
$d_{i,j,k}$: the activity progresses.

3.3.2.2. Action. Action space $\widehat{A}$ is a set of all the possible activity combinations (Eq.10). The size of $\widehat{A}$ is decided by the number of structure groups m. The action $\hat{a}$ represents the combination of conductible activities. For a major construction project, there might be thousands of activities. Generating all the activity combination requires high calculation resources. However, in a real-world construction project, the activities process such as piling or concrete casting must be continuous. These activities follow a fixed order, various from different projects. These operation sequences are parts of the projects' precedence constraints. When the structure groups are determined, the sub structures and activities belonging to the selected structure groups will be processed in a fixed order. Consequently, the action can be designed to only identify which structure group needs to be executed.

$$\widehat{A} = \begin{bmatrix} \widehat{a_1} \\ \vdots \\ \widehat{a_{2^m}} \end{bmatrix} = \begin{bmatrix} \overline{a_{1,1}} & \cdots & \overline{a_{1,m}} \\ \vdots & \ddots & \vdots \\ \overline{a_{2^m,1}} & \cdots & \overline{a_{2^m,m}} \end{bmatrix} \tag{10}$$

Where $\widehat{A}$ is a matrix with 2^m rows and m columns. Each column indicates the structure group at that index. Each row represents a combination of the structure group that should be processed. $\overline{a} = 1$ means the activity corresponding to its structure group needs to be processed, and $\overline{a} = 0$ means the opposite.

At each time step, the DRL agent chooses action $\hat{a}$ from a fixed action space $\widehat{A}$. This selection method is known as behaviour policy π. In DQN, an $\varepsilon - greedy$ policy is applied (Eq.11).

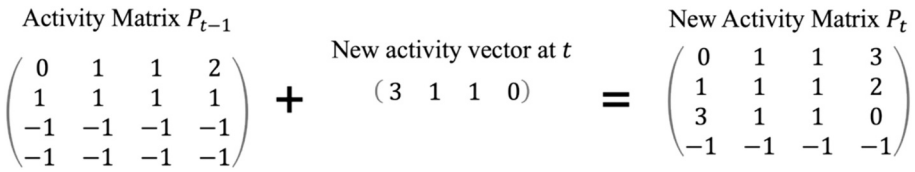


Fig. 5. Illustrative example of P

$$\hat{a} = \begin{cases} \underset{a}{argmax} Q(S, \hat{a}), \textit{in the probablity of } 1-\varepsilon \\ \textit{random}, \textit{in the probablity of } \varepsilon \end{cases} \tag{11}$$

The greedy policy tends to guide the agent to select the action which gives the maximum reward in the time step. At the beginning of the agent training, ε is set to be a larger value to let the agent explore the action space in a random pattern. This policy will prevent the agent to be trapped in local optimization and helps the neural networks have better generalization ability.

3.3.2.3. Reward shaping. The reward is the feedback from the environment after taking an action. The definition of the reward function determines the optimization direction of the DQN model. In this study, reward shaping is adopted to enhance the training efficiency [20]. The reward function consists of three parts. The first part is an end reward for completing the project (Eq.13). The second part is the reward directly related to the objective of shortening the total duration (Eq.12). The last part represents the reward for performing and completing activities during the training process (Eq.15, Eq.16).

The differences of reward design in various studies are listed in Table 2. Different from Soman and Molina-Solana's work [40], this reward shaping removes constraint violation reward which is benefit from the VAS mechanism. Since this study focus on shortening total duration, the cost reward in their work is also not considered. Considering the resources may allow parallel activities, we propose a composite reward combining on-going activities as well as completed activities and segments. Kedir et al. [25] only considered the reward as a negative value of time. Proposed study additionally considered activity completion as an independent reward, which encourages the agent choose multiple activities simultaneously.

$$RW_1 = 10000 \tag{13}$$

$$RW_2 = k \tag{14}$$

$$RW_3 = m_1 \times N_{complete\ activity} + m_2 \times N_{complete\ segment} \tag{15}$$

$$RW_4 = n \times N_{ongoing\ activities} \tag{16}$$

$$RW = \alpha RW_1 + RW_2 + RW_3 + RW_4 \tag{17}$$

Eq.13 represents the end reward for completing one episode. Eq.14 represents a penalty mechanism where k represents a negative real number. This negative reward aims to shorten the total duration by implying negative rewards. This reward encourages the agent to finish the scheduling completely as early as possible. Eq.15 indicates the reward for completing activities and segments in one time step. More completed activities and sub structures means the construction project is about to finish. Eq.16 indicates the reward for parallel activities; the agent is motivated to execute multiple activities simultaneously when resources are available. Eq.17 represents the total reward, $\alpha = 0$ means the construction is not finished and $\alpha = 1$ means the opposite. The value of k, m_1, m_2, n and its influence on the DRL performance will be discussed in following experiments.

Table 2
Reward design of different studies.

References	Reward Design			
	Time	Cost	Activity Completion	Constraint Violation
Soman et al. [40]	No	Yes	Yes	Yes
Kedir et al. [25]	Yes	No	No	No
Proposed study	Yes	No	Yes	No

3.3.2.4. Environment. The DRL environment responds to the agent's actions, generating new states and rewards. In this study, the interaction between the agent and environment is simulated via Python. When the DQN agent chooses a constraints-free action from the action space at the beginning of a timestep, the environment will first identify the state of every activity and resource. The completed activities and their assigned resources and subcontractors will be released. Then new activities will be started. Meanwhile, the reward will be calculated based on the activities' state and the number of completed activities in the timestep. At the end of the timestep, the reward and new state will be passed to the DQN agent for taking the next action. The pseudocode of the construction simulator is given in Algorithm. 1 The interaction between the classes and functions are illustrated in Fig. 6.

Algorithm. 1. Construction Simulator.

Algorithm 1: Construction Simulator

Input : Action $\hat{a}$
Output: New State S_{t+1}, Reward RW_t, End

Initialize: $RW_t = 0$, *End=False*;
while *not End* **do**
  Get Structure Group SG_a;
  for *sg in* SG_a **do**
    Get activities $\bar{a}$;
    for *a in* $\bar{a}$ **do**
      Get required resources R_{a-} ;
    end
    Add $\bar{a}$ to Activity Buffer A;
    for a' *in* A **do**
      Activity Duration +1;
      Calculate RW_2;
      if *Activity End* **then**
        Delete a' from A;
        Update Activity State P;
        Calculate RW_4;
        Get released resources R_{a+};
        Update Resource State R;
        Get Completed Structure Group sg and Segment r;
        Calculate RW_3;
        Update Structure State G';
      end
      else
        Update Activity State P;
        Calculate RW_4;
      end
      if *Project End* **then**
        Calculate RW_1;
        Reward $RW_t = RW_1+RW_2+RW_3+RW_4$;
        End Project;
      end
      else
        Return New State $S_{t+1} = [G'_{t+1}, P_{t+1}, R_{t+1}]$;
        Reward $RW_t = RW_2+RW_3+RW_4$;
      end
    end
  end
end
return *New State* S_{t+1}, *Reward* RW_t, *End*

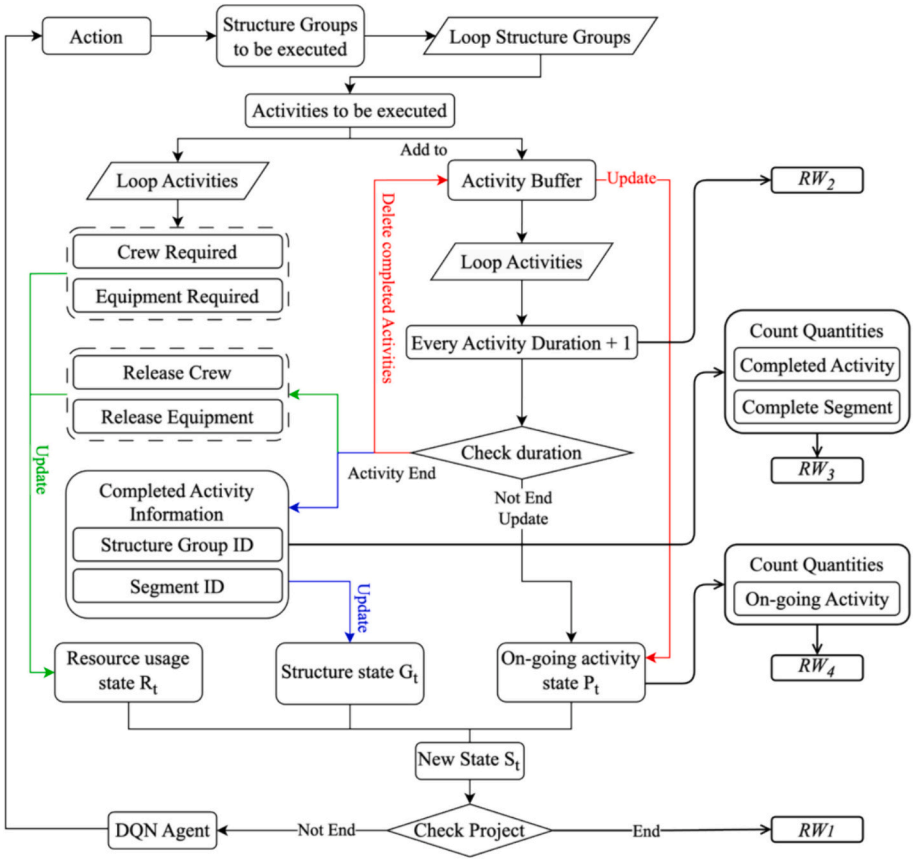


Fig. 6. Simulation environment diagram.

3.4. Valid action sampling

As mentioned in the action space design, a construction project with m structure groups has 2^m possible actions, among which most actions will lead to constraint violations. In some real construction projects, there may be tens or even hundreds of structure groups. Yet under different circumstances, only a few of them can be processed. At each timestep, the mask will provide a series of valid safe action combinations for the DQN agent according to the environment state S (Fig. 7).

To mask the invalid actions, a masking function is introduced (Eq.18):

$$\varphi(S,\widehat{a}) = \begin{cases} 1, if\ (S,\widehat{a})\ is\ verified\ safe \\ 0, otherwise \end{cases} \tag{18}$$

The masked action set $\widehat{A_\varphi}(S) = \{\widehat{a} | \varphi(S,\widehat{a}) = 1\}$ is defined for the environment state S. After the action masking, the agent will only choose valid actions from $\widehat{A_\varphi}$ under the $\varepsilon - greedy$ policy in Eq.11. After all the valid actions are identified, if the length of $\widehat{A_\varphi}$ is shorter than the expected length of A_μ, then the $\widehat{A_\varphi}$ will be padded with tokens. Otherwise, actions will be sampled from the $\widehat{A_\varphi}$. The pseudocode for the action masking is shown below (Algorithm. 2).

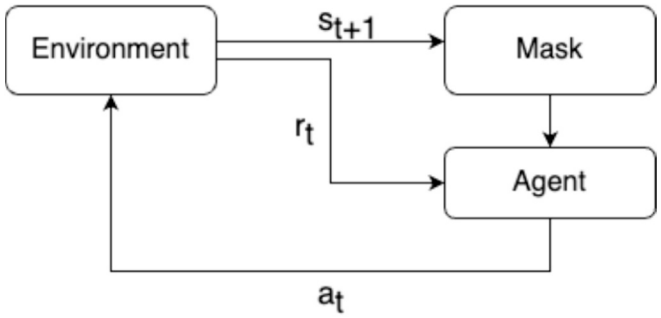


Fig. 7. Framework of action masking DRL.

At the decision point, the input to VAS is the initial action space and current state. The VAS will iterate every action in the environment. Thus, the resources state R and structure state S will be changed accordingly. A valid action must satisfy: 1) all resources remain no less than 0; and 2) the structure state does not exhibit a precedence violation. If a resource turns negative after executing an action, it indicates a violation of resource constraints. Precedence constraints in the environment mainly refer to the actions violating the constraints between structure groups. The VAS mechanism scrutinizes the node feature $\widehat{H}$ after the execution of an action. If an element within $\widehat{H}$ possesses a value distinct from -1, and there exists a preceding element with a value of -1, this condition is interpreted as a violation of the precedence constraint. Such iteration is carried out independently from the training process, the VAS simulation won't affect the training. After filtering out valid actions, VAS will randomly sample a fixed number of actions. If the number of valid actions is insufficient, the missing actions will be padded with dummy tokens to ensure the length of sampled actions is fixed.

Algorithm. 2. Valid Action Sampling.

Algorithm 2: Valid Action Sampling

Input : Action Space $\hat{A} = \begin{bmatrix} \widehat{a_1} \\ \vdots \\ \widehat{a_{2^m}} \end{bmatrix}$, State $S_t = [G_t, R_t, P_t]$, Sampled action length l_μ

Output: Valid Action Set A

```
A_φ = [ ];
for â in Â do
    Execute â in environment S_t;
    Return S_{t+1};
    Check precedence constraints of S_{t+1};
    Check resources constraints of S_{t+1};
    if No constraints violation then
        φ(S_t, â) = 1;
        Add â to A_φ
    end
end
if len(A_φ) ≤ l_μ then
    Pad A_φ to A_μ;
end
else
    Random sample l_μ action from A_φ;
end
return A_μ;
```

4. Case studies

4.1. Small-case study

The case selected in this study is based on a prefabricated construction project in Longdong Avenue, Shanghai, China. In prefabricated bridge construction, most of the above ground components are pre-cast. On-site construction structures include pile construction and platform casting. After that, the precast structures, including construction, columns, cover beams, and girders will be erected in sequence. Generally, a girder bridge includes two main parts: the substructure and the superstructure. The substructure includes piles, pile caps, piers, and pier caps. To improve the constructure quality, ICE Vibratory Hammer was used in this project, thus also fasten the piling process. The superstructure includes girders and decking. In this study, a three-span prefabricated girder bridge is taken for the case study; and decking is not considered. As the selected bridge consists of small prefabricated box girders, wet joints construction are also important processes. Thus, in this study, wet joint is considered one part of bridge superstructure. The bridge is divided into three structure groups. SG_1 represents the foundation; SG_2 represents the substructure; and SG_3 represents the superstructure. Each SG consists of various sub segments, the classification of which considers the different bridge segments and construction techniques are shown in Fig. 8.

The sub activities of each sub segments are listed in Table 3. The number of required labour and equipment are given. The activities will be assigned to different crews with different equipment. For example, a lifting crew will be assigned for pile lifting, pier lifting, pier cap lifting, and girder lifting with a walking crane. In this study, the construction process is divided into 28 kinds of activities with 12 different crews and 4 types of equipment. The total quantities of sub activities are 105. There are 4 Lifting crews and 4 Hammer & Piling crews; the quantities of other crews are 2. The quantities of Mobile Crane and Hammer are 4; and the quantities of other equipment are 2.

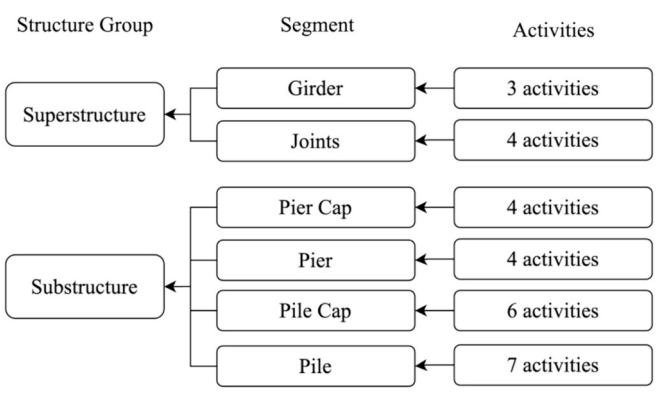


Fig. 8. Sub structures of the prefabricated bridge.

4.1.1. Assumptions

To model the processes of bridge construction, several assumptions are proposed. These assumptions provide an ideal condition of the construction site and operation regulations.

1) All the scheduled activities should be conducted according to the operation sequence and resource assignment.
2) All activities cannot be interrupted.
3) The assigned labour and equipment cannot be released before the activities are completed.
4) There is no time gap between the re-assignment of labour and equipment.
5) At time 0, all labour and equipment are available.
6) There is no waiting time for the prefabricated components to be transferred from the stacking area to the construction area.
7) The construction area is large enough to allow multiple parallel activities.
8) The processing time of activities is deterministic.

4.1.2. Model implementation

The GCN includes one input layer, two hidden layers, and one output layer. The numbers of input are the same as the length of $\widehat{H}$ and the number of output nodes is the same as the number of actions (size of $\widehat{A}$). There are 128 nodes and 64 nodes in the first and second hidden layers respectively. The output of the GCN is a vector with the length of 20, indicating the Q-values for the sampled valid actions. ReLU is chosen as the activation function. The batch size is 64, a 4-step frequency updating network and a 10,000-replay buffer size. The training was carried out with ε-greedy policy at an exploration factor of 0.95 and linearly decreasing to 0.01 at the end of the episode. Adam optimizer is adopted, and L2 Normalization is introduced during the network training. The DQN model is trained on a personal computer with an i7-11700k chip and an RTX 3090. The interpreter is based on Python 3.9.0, PyTorch 1.12.1, and Numpy 1.26.0. The GCN is established by using dgl 1.1.2 and the figures are generated by Matplotlib 3.8.0. To validate the effectiveness of GCN in state feature extraction, Multi-Layer Perceptron (MLP) is used as a comparison experiment. MLP is a commonly used feature extractor in the field of data processing. The MLP consists of 2 hidden layers, the numbers of two hidden dimensions are both 128. Activation function is ReLU and Adam optimizer is adopted.

10 different training settings are proposed to further evaluate the effectiveness of proposed GCN-based DQN framework (Table 4). S1-S5 represent the GCN-based algorithm and S6-S10 represent the MLP-based algorithm. S1 and S5 are carried out without reward shaping. And the other 8 training settings are aimed to discuss the impact of different value of rewards. RW_2 is the reward that directly related to the duration thus its value shall have impact on the training.

To evaluate the effect of the Valid Action Sampling (VAS) method, an initial ablation experiment is then conducted using a small case study for implementation. A standard DQN approach is utilized with same hyperparameters to those of the VAS for a fair comparison. The small case study featured 7 distinct structural groups, resulting in the DQN outputting a vector with 2^7Q-values for each potential action. To address the issue of the possibility of selecting invalid actions in normal DQN, any invalid choice by the agent results in a substantial negative reward, proportional to the number of constraints violated by the action.

Table 3
Activity details.

Segments	Sub Activity	Labour		Equipment		Duration (h)
		Crew	Number of Crew	Type	Number of Equipment	
Pile	Pile Lifting	Lifting Crew	1	Mobile Cranes	1	1
	Hammer Installing	Hammer & Pilling Crew	1	Mobile Cranes Hammer	1 1	1
	Piling	Hammer & Pilling Crew	1	Hammer	1	10
	Earth-moving	Earth Moving Crew	1	Auger	1	3
Pile Cap	Reinforcement Cage Sinking	Reinforcement Craft Crew	1	Mobile Cranes	1	1
	Concrete Casting	Concrete Casting Crew	1	Concrete Pump	1	2
	Pile End Trimming	Pile Trimming Crew	1			2
	Underlayment	Underlayment Crew	1	Concrete Pump	1	4
	Reinforcement cage tying	Reinforcement Craft Crew	1			4
	Mould assembling	Form Crew	1			2
	Concrete Casting	Concrete Casting Crew	1	Concrete Pump	1	2
	Curing					48
Pier	Mould removal	Form Crew	1			2
	Pier lifting	Lifting Crew	1	Mobile Cranes	1	2
	Junction Adjustment	Junction Crew	1	Mobile Cranes	1	2
	Junction Grouting	Junction Crew	1	Concrete Pump	1	2
	Curing					48
Pier Cap	Cap Lifting	Lifting Crew	1	Mobile Cranes	1	2
	Junction Adjustment	Junction Crew	1	Mobile Cranes	1	2
	Junction Grouting	Junction Crew	1	Concrete Pump	1	2
	Curing					48
Girder	Bearing Installing	Bearing Crew	1			2
	Girder Lifting	Lifting Crew	1	Mobile Cranes	1	2
	Girder Adjustment	Girder Crew	1	Mobile Cranes	1	2
Joints	Mould assembling	Form Crew	1			2
	Concrete Casting	Concrete Casting Crew	1	Concrete Pump	1	2
	Curing					48
	Mould removal	Form Crew	1			2

Table 4
Training settings.

Feature Extractor	Reward Shaping	RW_2				Settings
		−5	−25	−50	−60	
GCN		Y				S1
	Y	Y				S2
	Y		Y			S3
	Y			Y		S4
	Y				Y	S5
MLP		Y				S6
	Y	Y				S7
	Y		Y			S8
	Y			Y		S9
	Y				Y	S10

Such a reward structure is designed to incentivize the agent to avoid actions that break constraints.

4.1.3. Results for small-case study

The results consist of three parts: 1) comparison between GCN-based and MLP-based methods with different training settings; 2) comparison with traditional methods; and 3) result of ablation experiment.

4.1.3.1. Comparison between different training settings. The training processes of GCN-based DRL and MLP-based DRL under different settings are shown in Fig. 9 and Fig. 10. Blue curves represent the training reward curve; red curves represent the duration curve. The training of S1 and S6 show that the training of GCN and MLP without reward shaping fall into local optimum after 120 and 260 episodes respectively. The reward curves and duration curves of S1 and S2 show vibration around the local optimum. The training of S2 and S7 show that with reward shaping, the training processes of DQN show higher efficiency. GCN-based method reaches convergence in 30 episodes and MLP-based method reaches convergence around 35 episodes. Both methods find better solution than S1 and S5 with the total duration around 250 h. The training of S3 and S8 show that when $RW_2 = -25$, the training of GCN-based and MLP-based methods reach convergence at 20 and 30 episodes respectively. The training of S4 and S9 show that when RW_2 reaches −50, the efficiency of both GCN and MLP are slightly higher than $RW_2 = -25$. GCN-based method reached convergence at 15 episodes and MLP-based method convergent at 20 episodes. When $RW_2 = -60$ (S5 and S10), the training efficiency shows less improvement. Like $RW_2 = -50$, GCN-based method reached convergence at 15 episodes and MLP-based method convergent at 20 episodes.

Table 5 shows the training episodes and execution results of project duration after the training completed. Reward shaping greatly enhances the convergence performance of the DQN training and prevents the agent from falling into local optimum. The value of RW_2 has minor influence on the training and execution results. Overall, larger RW_2 can accelerate the training processes. For GCN-based method, larger RW_2 leads to better results. But for MLP-based method, the effect of the RW_2 is unstable. Thus, the value of RW_2 (k) is set to −60 as the results of S5 and S10 are better. The other coefficients of rewards are determined by similar processes. Through several trial-and-error experiments, the values of m_1, m_2, and n are set to 10, 50, and 2 accordingly. The values represent the significance of each reward for the agent and indicate the reward value the agent will receive upon fulfilling these conditions. It should be noted that the values can vary across different construction projects and are not unique.

4.1.3.2. Comparison with traditional algorithms. Table 6 shows the comparison of GCN-based DRL method with traditional method and GA method. The schedule made by experienced project scheduler manually leads to a 336 h duration while GA gave a better result with 287 h. DRL performed better than GA with 251 h duration. The GA algorithm adopted in this paper is proposed by Liu et al. [29]. The adopted algorithm used regret-based biased random sampling (RBRS) and serial schedule generation scheme (SSGS) to generate initial population. Each gene in the GA chromosome represents an individual sub activity. Thus, for the small case study, the length of the chromosome is the total

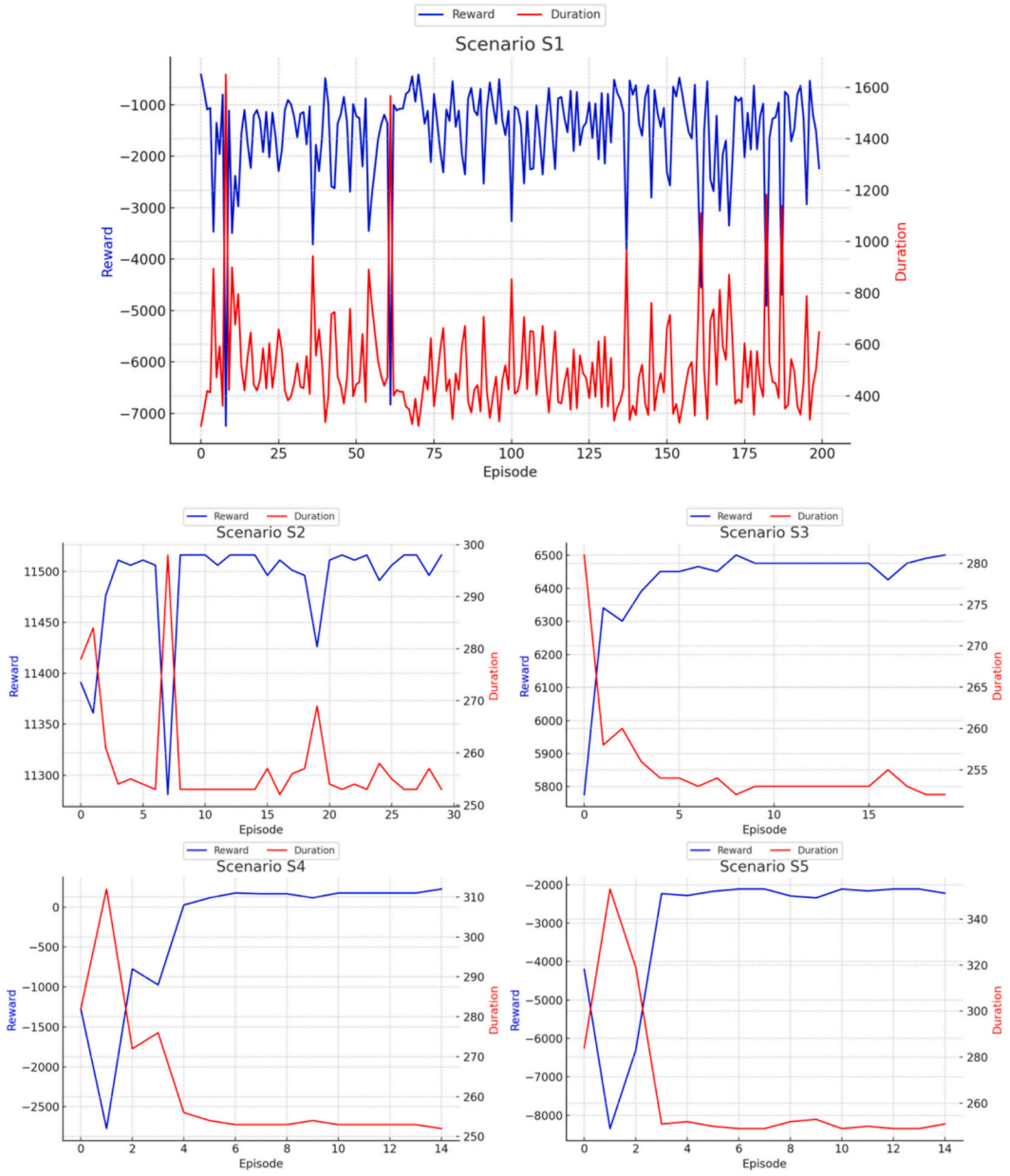


Fig. 9. Training curves of GCN-based DRL.

number of sub activities (105 activities in total). Evolution strategy consists of elite selection mechanism, two-point crossover, and linear decreasing probability-based mutation. The parameter settings of adopted GA are shown in Table 7. In this comparison, DRL method shows a stronger ability to find optimal solutions than CPM and GA methods with slightly faster CPU time.

4.1.3.3. Result of ablation experiment. The training process of a standard DQN without VAS is shown in Fig. 11. The bule curve represents smoothed moving-average reward curve; the red curve represents the duration curve during training. The training process runs 5000 episodes; and neither the reward curve nor duration curve shows the signs of convergence.

4.2. Large-case study

4.2.1. Case description

The larger case is derived from a real project in Meilong, Guangdong, China, where a prefabricated bridge was required to minimize the impact on the nearby environment. The total length of the selected bridge section is 390 m, which includes a 270-m main bridge section (9 spans of 30-m simple-supported beams) and two parallel ramp sections of 120 m each (4 spans of 30-m simple-supported beams for each ramp). To ensure a smooth transition between the main bridge and the ramp, the main bridge section includes a widening bridge segment (6 spans). The superstructures, substructures, and foundations of the three types of bridge section are different (Fig. 12 a, b, c). Compared with the small case study with 28 sub structures and 105 activities, this larger case study consists of 106 sub segments, which is a more complex construction project with 497 total activities. The construction method and equipment for the bridge are the same as in the previous case study. In addition, due to the larger scale of this case, the number of required equipment and manpower have also increased correspondingly (Table 8). There are 4 Lifting crews, 4 Hammer & Piling crews, 4 earth-moving crews, 4 reinforcement cage crews, 4 concrete crews, and 4 pile trimming crews; the quantities of other crews are 2. The quantities of all

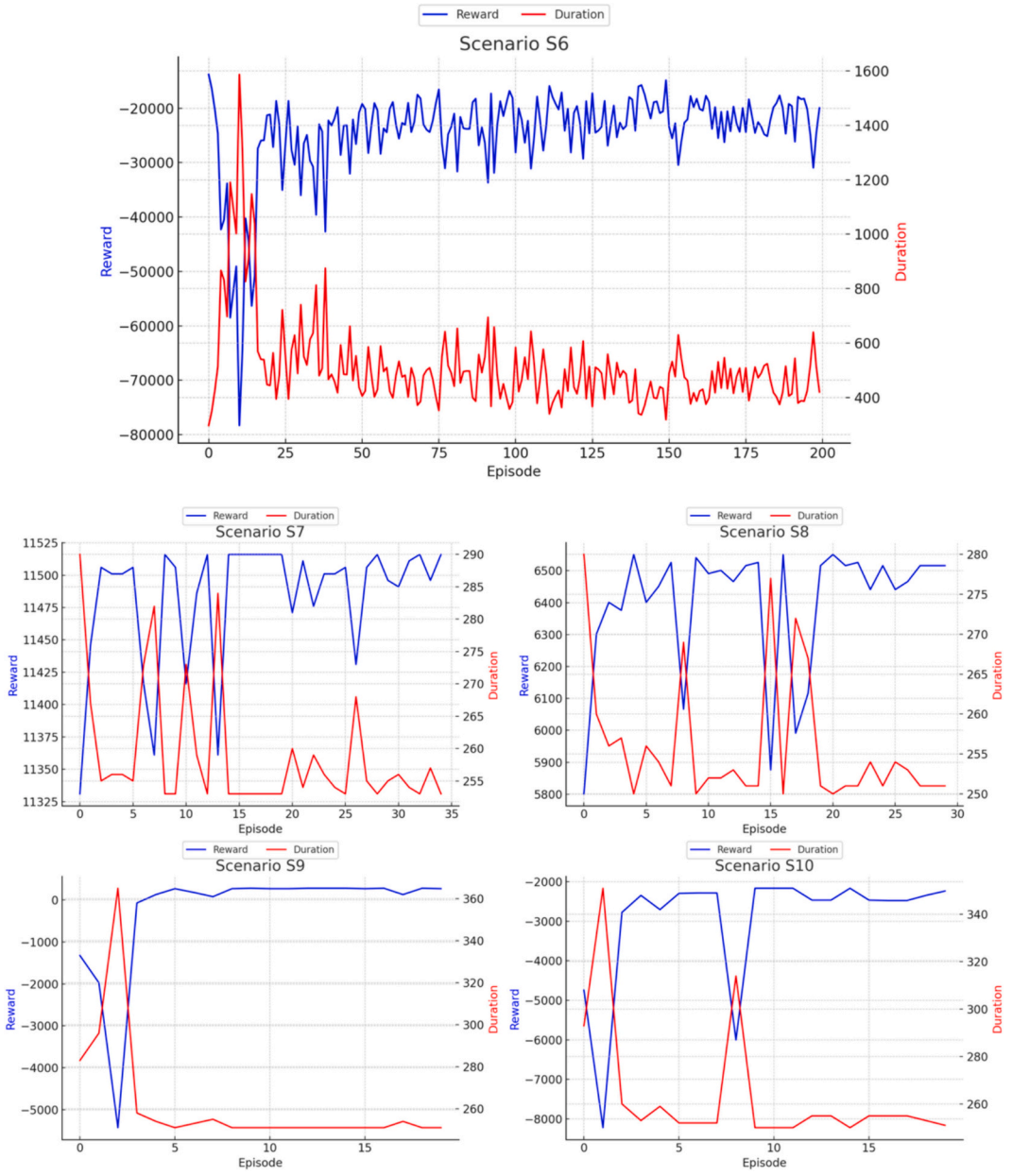


Fig. 10. Training Curves of MLP-based DRL.

Table 5
Comparison of the results.

	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Convergence Episode	130	30	20	15	15	260	35	30	20	20
Output Duration (h)	491	272	269	263	251	514	270	269	270	267

Table 6
Comparison with CPM and GA.

Method	Manual	GA	VAS DQN
Duration (h)	336	267	251
CPU Time	/	185 s	170 s

Table 7
Parameter settings of GA.

Population Size	500
Crossover Rate	0.8
Mutation Rate	0.2
TOP	0.15
BOT	0

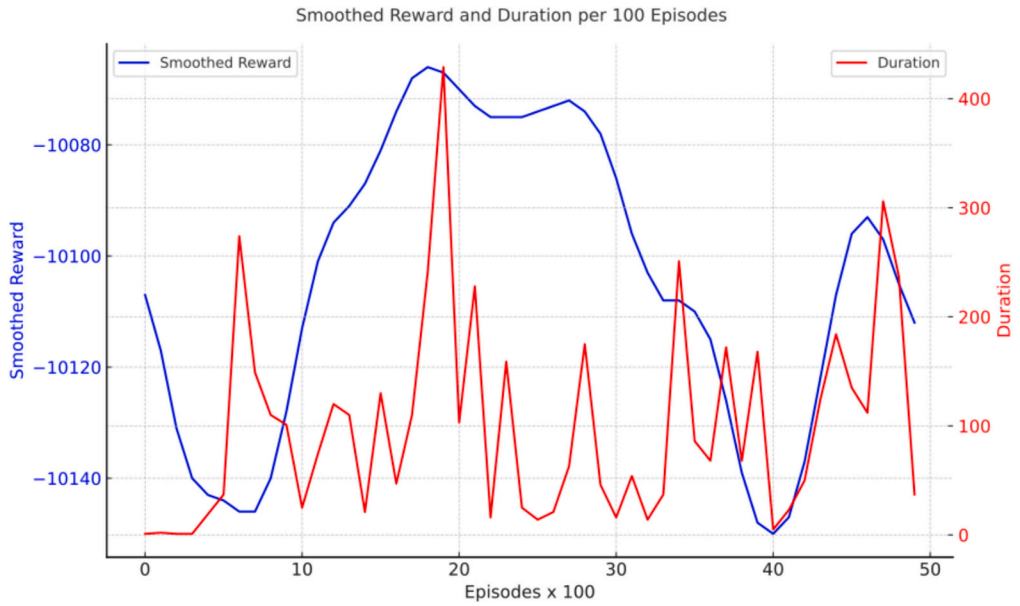


Fig. 11. Training curve without VAS.

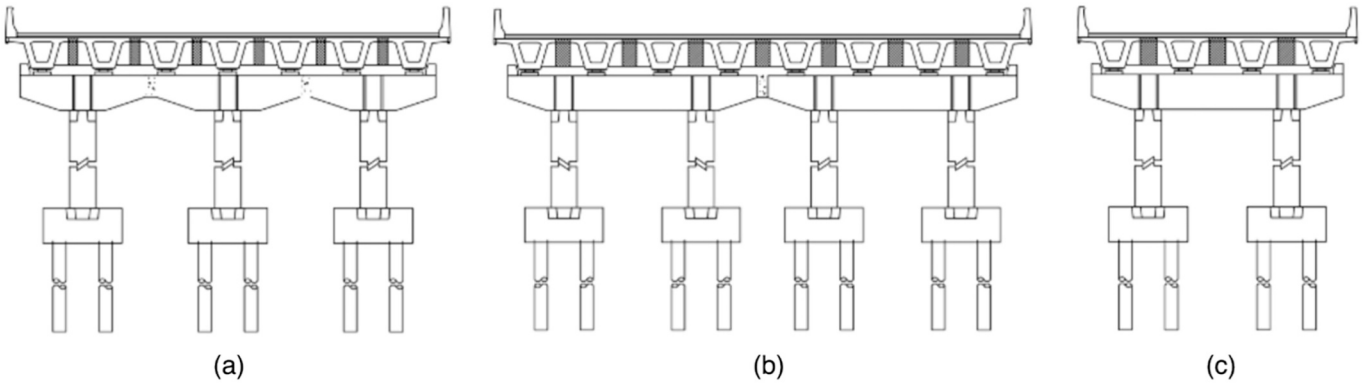


Fig. 12. The structures of different sections.

equipment are 4.

4.2.2. Results comparison

As the scale of the larger case study increases, the number of valid actions also increases accordingly. Therefore, the size of the sampling will affect the training of the DQN. To verify the impact of the sampling size on the training process, this study conducted three experiments with sampling sizes of 50, 80, and 100, respectively. The training processes of different sampling size are illustrated in Fig. 13 accordingly. The blue curve indicates reward data; and the orange curve indicates the smoothed moving-average which shows the training trend. The analysis reveals that a smaller sample size (50) results in a protracted exploration phase, as indicated by the average reward remaining below the threshold of 6000 for approximately the first 120 episodes. This phenomenon is attributable to the inadequate action space sampling, necessitating extended episodes for comprehensive exploration. Conversely, larger sample sizes (80 and 100) facilitate a more expedient exploration phase, evidenced by quicker attainment of stable and higher rewards. Notably, the agent trained with a sample size of 100 exhibits superior stability in its training process compared to its counterpart with a sample size of 80. This stability is manifest post−100 episodes, where the rewards not only consistently surpass the 6000 mark but also display reduced volatility. This observation underscores the hypothesis that increased sample sizes not only abbreviate the exploration phase but also enhance the learning algorithm's stability, leading to a more efficient and effective training process.

Due to the random sampling of valid actions, the output of the trained agent is often fluctuating. Therefore, the results for comparison are taken as the mean value of 20 outputs from the trained agent and GA. The mean values of the algorithms are rounded to the nearest integer (Table 9). It should be noted that the original manual schedule is a rough master plan used for determining milestones, requiring the project to be completed in 50 days. Given the practical working hours for this project are about 10 h per day, the duration provided by traditional methods translates to 500 h.

4.3. Rescheduling experiments

To validate the efficacy of the DQN method's capacity for rescheduling, a series of experiments were conducted with varying initial resource configurations. In these rescheduling experiments, all labour and equipment resources were adjusted within a range of −1 to +4, based on their original quantities. The experimental evaluation conducted a comparative analysis of the optimization results achieved by the pre-trained DQN model, GA, and a retrained DQN across various initial resource conditions. The outcomes of the rescheduling experiments are detailed in Table 10. Within different resource configurations, the GA exhibited longer average durations and CPU times compared to

Table 8
Larger case activity details.

Sub structure	Sub Action	Crew				Equipment				Duration (h)		
			Type a	Type b	Type c		Type a	Type b	Type c	Type a	Type b	Type c
Pile	Pilling	Lifting Crew	1	2	1	Walking Cranes	1	2	1	16	18	12
		Pilling Crew	1	2	1	Hammer	1	2	1			
	Earth-moving	Earth Moving Crew	1	2	1	Excavator	1	2	1	2	3	3
	Rebar Cage Sinking	Rebar Cage Crew	1	1	1	Walking Cranes	1	2	1	2	2	1
	Concrete Casting	Concrete Casting Crew	1	1	1	Concrete Pump	1	2	1	2	3	2
	Pile End Trimming	Pile Trimming Crew	1	1	1					2	3	2
Pile cap	Underlayment	Underlayment Crew	1	2	1	Concrete Pump	1	2	1	4	4	4
	Rebar cage tying	Rebar Craft Crew	1	1	1					4	6	4
	Mould assembling	Form Crew	1	1	1					2	3	2
	Concrete Casting	Concrete Casting Crew	1	1	1	Concrete Pump	2	2	1	3	4	2
	Curing									48	48	48
	Mould removal	Form Crew	1	2	1					2	2	2
Pier	Pier lifting	Lifting Crew	1	2	1	Walking Cranes	1	2	1	2	3	2
	Junction Adjustment	Junction Crew	1	2	1	Walking Cranes	1	2	1	2	2	2
	Junction Grouting	Junction Crew	1	1	1	Concrete Pump	1	1	1	2	2	2
	Curing									48	48	48
Pier Cap	Cap Lifting	Lifting Crew	1	2	1	Walking Cranes	1	2	1	2	2	2
	Junction Adjustment	Junction Crew	1	2	1	Walking Cranes	1	2	1	2	2	2
	Junction Grouting	Junction Crew	1	1	1	Concrete Pump	1	1	1	2	2	2
	Curing									48	48	48
Girder	Bearing Installing	Bearing Crew	1	2	1	Walking Cranes	1	2	1	2	2	2
	Girder Lifting	Lifting Crew	1	2	1	Walking Cranes	1	2	1	4	6	2
	Girder Adjustment	Girder Crew	1	1	1					2	2	2
Joints	Mould assembling	Form Crew	1	2	1					2	2	2
	Concrete Casting	Concrete Casting Crew	1	2	1	Concrete Pump	1	2	1	3	3	2
	Curing									48	48	48
	Mould removal	Form Crew	1	1	1					2	2	2

the retrained DQN. Although the schedules produced by the pre-trained DQN were not as optimal as those generated by the retrained DQN, the time required was shorter, and the results are better than those recalculated by the GA.

5. Discussion

In previous sections, the proposed VAS DQN-based Constrained Scheduling Method is validated via case studies in precast bridge construction. Different training settings are adopted to identify the effectiveness of the proposed reward shaping methods. An ablation experiment is carried out to validate the VAS mechanism. Different scheduling methods are implemented and compared to generate schedules for the studied case.

5.1. Discussion on different training settings

The proposed GCN-based method is first compared with MLP-based method under different training settings. The results indicate that GCN-based method shows superiority in training than MLP-based method with proper training settings. Compared with non-reward-shaping methods, methods with reward shaping shows higher efficiency in both training process and convergence. In the real-word precast bridge project, the rewards provided to the agent are sparse, meaning that the agent receives feedback only occasionally or at the end of an episode. Sparse rewards can lead to slow learning and difficulties in exploration, as the agent might not receive enough information to understand which actions are leading it closer to the desired outcome. Reward shaping can provide additional, intermediate rewards to the agent during its exploration, helping it learn faster by providing more informative feedback. Meanwhile, reward shaping can direct the learning process towards favourable behaviour by assigning greater rewards to actions that contribute positively to accomplishing the overall objective. By shaping the rewards, the agent is encouraged to focus on important aspects of the task and avoid unnecessary exploration of irrelevant actions, thereby hastening the convergence towards optimal strategies.

5.2. Discussion on scheduling methods comparison

The proposed method is further compared with traditional methods and shows better performance in minimizing the total duration. The GA's suboptimal performance is mainly attributed to its convergence to a local optimum, primarily caused by a lack of diversity in the initial population. While the GA heavily relies on the initial population, this issue does not affect DRL as it is not population-based. The action masking prevents the DRL from choosing invalid actions and the reward shaping guides the agent to choose optimal solutions. Consequently, each action in DRL effectively leverages the constraint checking environment's capabilities, thus lead to its ability in fast reaching convergence and finding global optimum. The requirement of GA for iteration optimization necessitates their re-optimization when confronted with different initial resource configurations. In contrast, the DQN benefits from the generalization ability of neural networks, enabling it to instantly provide sub-optimal schedules even when the initial resource configurations vary. These schedules can serve as references for real-time evaluation.

The variability in VAS DQN outputs can be attributed to two main factors: 1) inherent randomness of the DQN agent and 2) the stochastic nature of valid action sampling. In the context of a small case study, the relative simplicity and lower complexity of the case mean that the agent

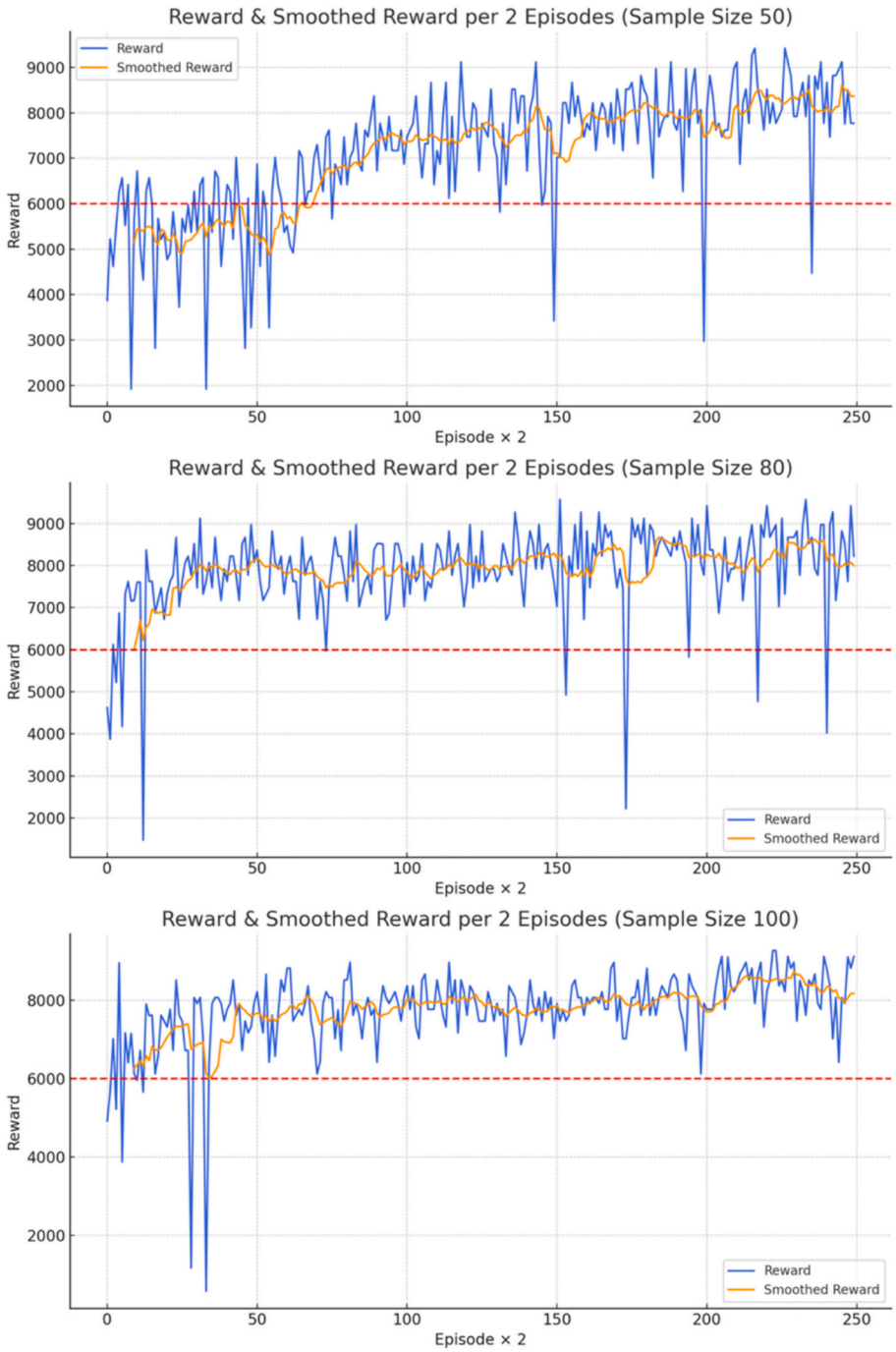


Fig. 13. Training curves of different sample size.

Table 9
Comparison on larger case.

Method	Manual	GA	VAS DQN		
Sample size	/	/	100	80	50
Best schedule (h)	/	394	372	377	394
Worst schedule (h)	/	419	385	398	415
Mean (h)	500	410	378	392	403
Average CPU Time	/	1283s	387 s	391 s	385 s

is not significantly affected by this randomness, resulting in stable output. However, the influence of randomness is particularly pronounced in larger case studies. Even with a sampling size of 100, there is a slight fluctuation in the agent's output. While the sampling sizes of 100 and 80 do not differ substantially in producing the best schedule, the latter's worst schedule and mean performance are significantly inferior to the formers. Therefore, for complex major construction projects, appropriately adjusting the sampling size can lead to better outcomes.

5.3. Discussion on ablation experiment

A significant disparity exists between the performances of the standard DQN and the VAS DQN. Over an extended period of training spanning 5000 episodes, the reward curve for the standard DQN does not demonstrate signs of improvement. Additionally, the duration curve indicates that the majority of episodes terminate prematurely due to

Table 10
Rescheduling results comparison.

Algorithm		Resource Configurations				
		-1	+1	+2	+3	+4
GA	Average Duration (h)	437	409	398	392	388
	Average CPU time	1397s	1351s	1327s	1305s	1311s
Retrained DQN	Average Duration (h)	386	371	368	363	357
	Average CPU time	390 s	391 s	387 s	385 s	389 s
Pre-trained DQN	Average Duration (h)	401	389	383	380	380
	Average CPU time	6 s	6 s	6 s	6 s	6 s

violations of constraints. This issue arises because within the 2^7 action space, only a small fraction of actions is free from causing constraint violations. The agent is tasked with exploring and updating the Q-value for each action, a challenging feat that hinders learning a complete project trajectory from start to finish, thus reducing training efficiency. Fundamentally, the essence of DQN, a tabular DRL method, is for the agent to determine the value of each action in various states. Thus, excluding invalid actions directly during training effectively equates to assigning them negligible Q-values, reducing their detrimental influence on the agent's training and exploration. Consequently, VAS is theoretically fitted with the foundational principles of DQN.

6. Limitation

This study has some limitations. First, the proposed method is limited to shortening the project duration without considering minimizing the project costs. The DQN features are designed for the optimization of single objective under heterogeneous constraints. Multi-agent or multi-objective optimization is not performed in this study. The proposed method will be expanded in future work to represent more complex projects with various project duration distributions, multiple subcontractors, and various resources. Additionally, the suggested method will be enhanced to carry out multi-objective optimisation with additional constraints, such as time, cost, and quality. Second, this work only introduces a simple version of DQN to solve construction scheduling problem. Future work should focus on implementing improved DQN methods such as Double DQN (Van Hasselt et al., 2015), Duelling DQN (Wang et al., 2016), and Rainbow DQN (Hessel et al., 2018). The reward shaping mechanism is shallowly discussed and its full potential is not developed. Future work should study on more types of reward shaping tricks and try to conclude a general paradigm that can be applied in more construction scheduling problems. Third, the case studies in this paper only focus on the bridge construction; and the proposed assumptions are too idealistic. The external factors such weather, the uncertainties such as accidents are not considered. Future work should explore the DRL's capacity in handling the real-world complexity or other form of construction projects. Finally, the rescheduling experiments were limited to examining various resource configurations, and the resulting schedules generated by the pre-trained DQN were observed to be sub-optimal compared to those produced by a retrained DQN. Future research should focus on enhancing the rescheduling capabilities of pre-trained DQN. Additionally, there is a need to develop new algorithmic architectures that imbue pre-trained DQN with transferability, enabling their application across diverse construction projects.

7. Conclusion

The VAS DQN-based Constrained Scheduling Method developed in this paper uses DRL to generate constraint-free dynamic schedules to aid on-site planning. A DQN model with VAS was used to model the construction scheduling problem. The scheduling problem aims to shorten the total project duration under constraints. VAS is proposed and adopted in the DQN training, which narrowed the action space for the agent to choose actions. Reward shaping is applied which enhanced the performance of the DQN and prevented the network from local optimum. The proposed method can generate schedules better than traditional method and GA.

The main contribution of this paper is to explore the application of reinforcement learning to real construction scheduling problems with complex constraints and propose an appliable VAS DQN architecture for large construction projects. Compared to traditional methods, the automatic generation of detailed schedules will help managers quickly determine the allocation of resources and labour. The action masking dramatically enhances the efficiency of DQN training by reducing the agent's exploring time, which provides possibilities for applying the method in large-scale projects. Proper reward shaping method is introduced and discussed the influence on convergent the DQN model. GCN performs better than MLP in convergence speed and execution results. The rescheduling ability of proposed method indicates that pre-trained DQN could be used to make fast rescheduling under various resource configurations.

This study is motivated by state-of-the-art research in the field of automatic construction scheduling and planning. Especially few studies have been made on integrating DRL and scheduling with heterogenous constraints. Researchers in Construction Management can extend the proposed method in different cases and develop different methods for enhancing the effectiveness of project scheduling. Future method should possess generalization capabilities, allowing it to schedule a wide range of projects across all aspects of the construction industry without changing the basic algorithmic structure. Future methods can integrate with the Building Information Modelling (BIM) and the Internet of Things (IoT) to explore additional applications in lean construction. Researchers in construction material manufacturing will also find this method useful as the construction schedule determines when a certain type of material is needed on-site. Solid construction project benchmarks should also be developed in the future. The benchmark should contain various type of construction projects with their optimal or near-optimal schedules. Such benchmark could enhance the application of artificial intelligence methods in this field. Finally, Mechanical, Electrical, and Plumbing (MEP) engineers can also extend this method to help make schedules after the construction is completed. In sum, the DRL-based decision methods still have a wide scope of research and application in the field of construction scheduling.

CRediT authorship contribution statement

Yuan Yao: Writing – review & editing, Writing – original draft, Formal analysis, Conceptualization. **Vivian W.Y. Tam:** Writing – review & editing, Writing – original draft, Resources, Project administration, Funding acquisition, Conceptualization. **Jun Wang:** Supervision. **Khoa N. Le:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Funding acquisition. **Anthony Butera:** Writing – review & editing, Writing – original draft, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

The authors wish to acknowledge the financial support from the Australian Research Council (ARC), Australian Government (No: DP200100057, IH200100010; FT220100017).

References

[1] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Commun. 7 (1994) 39–59, https://doi.org/10.3233/AIC-1994-7104.
[2] T. Ahmed, S.M. Hossain, M.A. Hossain, Reducing completion time and optimizing resource use of resource-constrained construction operation by means of simulation modeling, Int. J. Constr. Manag. 21 (4) (2021) 404–415, https://doi.org/10.1080/15623599.2018.1543109.
[3] F. Amer, Y. Jung, M. Golparvar-Fard, Construction schedule augmentation with implicit dependency constraints and automated generation of lookahead plan revisions, Autom. Constr. 152 (2023) 1–13, https://doi.org/10.1016/j.autcon.2023.104896. 104896.
[4] F. Amer, Y. Jung, M. Golparvar-Fard, Transformer machine learning language model for auto-alignment of long-term and short-term plans in construction, Autom. Constr. 132 (2021) 1–14, https://doi.org/10.1016/j.autcon.2021.103929. 103929.
[5] F. Amer, H.Y. Koh, M. Golparvar-Fard, Automated methods and systems for construction planning and scheduling: Critical review of three decades of research, J. Constr. Eng. Manag. 147 (7) (2021) 1–17, https://doi.org/10.1061/(asce)co.1943-7862.0002093. 03121002.
[6] C.P. Andriotis, K.G. Papakonstantinou, Managing engineering systems with large state and action spaces through deep reinforcement learning, Reliab. Eng. Syst. Saf. 191 (2019) 1–17, https://doi.org/10.1016/j.ress.2019.04.036. 106483.
[7] V. Asghari, Y.Y. Wang, A.J. Biglari, S.C. Hsu, P.B. Tang, Reinforcement learning in construction engineering and management: A review, J. Constr. Eng. Manag. 148 (11) (2022) 1–23, https://doi.org/10.1061/(asce)co.1943-7862.0002386. 03122009.
[8] Y. Bai, Y. Zhao, Y. Chen, L. Chen, Designing domain work breakdown structure (DWBS) using neural networks, Advances in Neural, Networks 5553 (2009) 1146–1153, https://doi.org/10.1007/978-3-642-01513-7_127.
[9] O.H. Bettemir, R. Sonmez, Hybrid genetic algorithm with simulated annealing for resource-constrained project scheduling, J. Manag. Eng. 31 (5) (2015) 1–8, https://doi.org/10.1061/(asce)me.1943-5479.0000323. 04014082.
[10] L. Bianco, M. Caramia, S. Giordani, Resource levelling in project scheduling with generalized precedence relationships and variable execution intensities, OR Spectr. 38 (2) (2016) 405–425, https://doi.org/10.1007/s00291-016-0435-1.
[11] M.-Y. Cheng, D. Prayogo, D.-H. Tran, Optimizing multiple-resources leveling in multiple projects using discrete symbiotic organisms search, J. Comput. Civ. Eng. 30 (3) (2016) 1–9, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000512. 04015036.
[12] S. Christodoulou, Scheduling Resource-Constrained Projects with Ant Colony Optimization Artificial Agents, J. Comput. Civ. Eng. 24 (1) (2010) 45–55, https://doi.org/10.1061/(ASCE)0887-3801(2010)24:1(45).
[13] H. Ding, C. Zhuang, J. Liu, Extensions of the resource-constrained project scheduling problem, Autom. Constr. 153 (2023) 1–26, https://doi.org/10.1016/j.autcon.2023.104958. 104958.
[14] K. El-Rayes, D.H. Jun, Optimizing resource leveling in construction projects, J. Constr. Eng. Manag. 135 (11) (2009) 1172–1180, https://doi.org/10.1061/(ASCE)CO.1943-7862.0000097.
[15] M. Erdal, R. Kanit, Scheduling of construction projects under resource-constrained conditions with a specifically developed software using genetic algorithms, Tehnicki Vjesnik-Technical Gazette 28 (4) (2021) 1362–1370, https://doi.org/10.17559/tv-20200305101811.
[16] A. Fischer Martin, F. Aalami, Scheduling with computer-interpretable construction method models, J. Constr. Eng. Manag. 122 (4) (1996) 337–347, https://doi.org/10.1061/(ASCE)0733-9364(1996)122:4(337.
[17] E.N. Goncharov, V.V. Leonov, Genetic algorithm for the resource-constrained project scheduling problem, Autom. Remote. Control. 78 (6) (2017) 1101–1114, https://doi.org/10.1134/S0005117917060108.
[18] A. Gupta, Y. Badr, A. Negahban, R.G. Qiu, Energy-efficient heating control for smart buildings with deep reinforcement learning, J. Build. Eng. 34 (2021) 1–15, https://doi.org/10.1016/j.jobe.2020.101739. 101739.
[19] Y. Hong, H. Xie, E. Agapaki, I. Brilakis, Graph-based automated construction scheduling without the use of BIM, J. Constr. Eng. Manag. 149 (2) (2023) 1–15, https://doi.org/10.1061/JCEMD4.COENG-12687. 05022020.
[20] Y. Hu, W. Wang, H. Jia, Y. Wang, Y. Chen, J. Hao, F. Wu, C. Fan, Learning to utilize shaping rewards: A new approach of reward shaping, Adv. Neural Inf. Proces. Syst. 33 (2020) 15931–15941, https://doi.org/10.48550/arXiv.2011.02669.
[21] Z. Hua, Z. Liu, L. Yang, L. Yang, Improved genetic algorithm based on time windows decomposition for solving resource-constrained project scheduling problem, Autom. Constr. 142 (2022) 1–14, https://doi.org/10.1016/j.autcon.2022.104503. 104503.
[22] S. Huang, S. Ontañón, A closer look at invalid action masking in policy gradient algorithms, arXiv preprint (2020) 1–10, https://doi.org/10.48550/arXiv.2006.14171, arXiv:2006.14171.
[23] C. Jiang, X. Li, J.-R. Lin, M. Liu, Z. Ma, Adaptive control of resource flow to optimize construction work and cash flow via online deep reinforcement learning, Autom. Constr. 150 (2023) 1–20, https://doi.org/10.1016/j.autcon.2023.104817. 104817.
[24] L.P. Kaelbling, M.L. Littman, A.W. Moore, Reinforcement learning: A survey, J. Artif. Intell. Res. 4 (1996) 237–285, https://doi.org/10.48550/arXiv.cs/9605103.
[25] N.S. Kedir, S. Somi, A.R. Fayek, P.H.D. Nguyen, Hybridization of reinforcement learning and agent-based modeling to optimize construction planning and scheduling, Autom. Constr. 142 (2022) 1–18, https://doi.org/10.1016/j.autcon.2022.104498. 104498.
[26] J.L. Kolodner, An introduction to case-based reasoning, Artif. Intell. Rev. 6 (1) (1992) 3–34, https://doi.org/10.1007/BF00155578.
[27] I. Kurinov, G. Orzechowski, P. Hamalainen, A. Mikkola, Automated excavator based on reinforcement learning and multibody system dynamics, IEEE Access 8 (2020) 213998–214006, https://doi.org/10.1109/ACCESS.2020.3040246.9268069.
[28] J.C.W. Lin, Q. Lv, D.H. Yu, G. Srivastava, C.H. Chen, Optimized scheduling of resource-constraints in projects for smart construction, Inf. Process. Manag. 59 (5) (2022) 1–15, https://doi.org/10.1016/j.ipm.2022.103005. 103005.
[29] J. Liu, Y. Liu, Y. Shi, J. Li, Solving resource-constrained project scheduling problem via genetic algorithm, J. Comput. Civ. Eng. 34 (2) (2020) 1–10, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000874. 04019055.
[30] W.L. Liu, J.W. Zhang, W.J. Li, Heuristic methods for finance-based and resource-constrained project scheduling problem, J. Constr. Eng. Manag. 147 (11) (2021) 1–15, https://doi.org/10.1061/(asce)co.1943-7862.0002174. 04021141.
[31] Y. Liu, J.J. Dong, L. Shen, A conceptual development framework for prefabricated construction supply chain management: An integrated overview, Sustainability 12 (5) (2020) 1–29, https://doi.org/10.3390/su12051878.1878.
[32] Z.L. Ma, S.Y. Li, Y. Wang, Z.Q. Yang, Component-level construction schedule optimization for hybrid concrete structures, Autom. Constr. 125 (2021) 1–13, https://doi.org/10.1016/j.autcon.2021.103607. 103607.
[33] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533, https://doi.org/10.1038/nature14236.
[34] T.M. Moerland, J. Broekens, A. Plaat, C.M. Jonker, A unifying framework for reinforcement learning and planning, Frontiers in Artificial, Intelligence 5 (2022) 1–25, https://doi.org/10.3389/frai.2022.908353. 908353.
[35] A. Neumann, A. Hajji, M. Rekik, R. Pellerin, Integrated planning and scheduling of engineer-to-order projects using a Lamarckian Layered Genetic Algorithm, Int. J. Prod. Econ. 267 (2024) 1–17, https://doi.org/10.1016/j.ijpe.2023.109077.109077.
[36] E. Ratajczak-Ropel, Experimental evaluation of agent-based approaches to solving multi-mode resource-constrained project scheduling problem, Cybern. Syst. 49 (5–6) (2018) 296–316, https://doi.org/10.1080/01969722.2017.1418269.
[37] S. Renard, B. Corbett, O. Swei, Minimizing the global warming impact of pavement infrastructure through reinforcement learning, Resour. Conserv. Recycl. 167 (2021) 1–13, https://doi.org/10.1016/j.resconrec.2020.105240. 105240.
[38] K.M. Sallam, R.K. Chakrabortty, M.J. Ryan, A reinforcement learning based multi-method approach for stochastic resource constrained project scheduling problems, Expert Syst. Appl. 169 (2021) 1–19, https://doi.org/10.1016/j.eswa.2020.114479.114479.
[39] C. Schiwindt, J. Zimmermann, Handbook on Project Management and Scheduling vol.1, Springer International Publishing, 2015. ISBN3319054422, https://link-springer-com.ezproxy.uws.edu.au/book/10.1007/978-3-319-05443-8.
[40] R.K. Soman, M. Molina-Solana, Automating look-ahead schedule generation for construction using linked-data based constraint checking and reinforcement learning, Autom. Constr. 134 (2022) 1–16, https://doi.org/10.1016/j.autcon.2021.104069. 104069.
[41] R.S. Sutton, A.G. Barto, Reinforcement learning: An introduction, MIT press, Cambridge, Massachusetts, 2018. ISBN0262352702, https://mitpress.mit.edu/9780262039246/reinforcement-learning/.
[42] M. Taghaddos, A. Mousaei, H. Taghaddos, U. Hermann, Y. Mohamed, S. AbouRizk, Optimized variable resource allocation framework for scheduling of fast-track industrial construction projects, Autom. Constr. 158 (2024) 1–24, https://doi.org/10.1016/j.autcon.2023.105208. 105208.
[43] C. Verbeeck, V. Van Peteghem, M. Vanhoucke, P. Vansteenwegen, E.-H. Aghezzaf, A metaheuristic solution approach for the time-constrained project scheduling problem, OR Spectr. 39 (2) (2017) 353–371, https://doi.org/10.1007/s00291-016-0458-7.
[44] S. Wei, Y. Bao, H. Li, Optimal policy for structure maintenance: A deep reinforcement learning framework, Struct. Saf. 83 (2020) 1–13, https://doi.org/10.1016/j.strusafe.2019.101906. 101906.
[45] L.L. Xie, Y.J. Chen, R.D. Chang, Scheduling optimization of prefabricated construction projects by genetic algorithm, Appl Sci Basel 11 (12) (2021) 1–22, https://doi.org/10.3390/app11125531. 5531.
[46] L. Yao, Q. Dong, J. Jiang, F. Ni, Deep reinforcement learning for long-term pavement maintenance planning, Comp Aided Civil Infrastruct. Eng. 35 (11) (2020) 1230–1245, https://doi.org/10.1111/mice.12558.
[47] Y.J. Ye, D.W. Qiu, H.Y. Wang, Y. Tang, G. Strbac, Real-time autonomous residential demand response management based on twin delayed deep deterministic policy gradient learning, Energies 14 (3) (2021) 1–22, https://doi.org/10.3390/en14030531. 531.
[48] Y.S. Yuan, S.D. Ye, L. Lin, M.S. Gen, Multi-objective multi-mode resource-constrained project scheduling with fuzzy activity durations in prefabricated

building construction, Comput. Ind. Eng. 158 (2021) 1–16, https://doi.org/10.1016/j.cie.2021.107316. 107316.
[49] Y. Zhang, X. Li, Y. Teng, G.Q. Shen, S. Bai, A heuristic rule adaptive selection approach for multi-work package project scheduling problem, Expert Syst. Appl. 238 (2024) 1–21, https://doi.org/10.1016/j.eswa.2023.122092. 122092.
[50] J. Zhou, P.E.D. Love, X. Wang, K.L. Teo, Z. Irani, A review of methods and algorithms for optimizing construction scheduling, J. Oper. Res. Soc. 64 (8) (2013) 1091–1105, https://doi.org/10.1057/jors.2012.174.